AdaptOrch: Task-Adaptive Multi-Agent Orchestration in the Era of LLM Performance Convergence

Geunbin Yu
Department of Artificial Intelligence, Korea National Open University
ict03@rfems.com
ORCID: 0009-0006-2879-9514

February 2026

Abstract

As large language models (LLMs) from diverse providers converge toward comparable benchmark performance, the traditional paradigm of selecting a single best model per task yields diminishing returns. We argue that **orchestration topology**—the structural composition of how multiple agents are coordinated, parallelized, and synthesized—now dominates system-level performance over individual model capability. We present AdaptOrch, a formal framework for **task-adaptive multi-agent orchestration** that dynamically selects among four canonical topologies (parallel, sequential, hierarchical, and hybrid) based on task dependency graphs and empirically derived domain characteristics. Our framework introduces three key contributions: (1) **Performance Convergence Scaling Law**, formalizing conditions under which orchestration selection outweighs model selection; (2) **Topology Routing Algorithm** that maps task decomposition DAGs to optimal orchestration patterns in $O(|V| + |E|)$ time; and (3) **Adaptive Synthesis Protocol** with provable termination guarantees and heuristic consistency scoring for parallel agent outputs. We validate AdaptOrch across coding (SWE-bench), reasoning (GPQA), and retrieval-augmented generation tasks, demonstrating that topology-aware orchestration achieves 12–23% improvement over static single-topology baselines, even when using identical underlying models. Our results establish orchestration design as a first-class optimization target independent of model scaling.

Keywords: multi-agent systems, LLM orchestration, task-adaptive routing, parallel agent execution, performance convergence

1 Introduction

The landscape of large language models in early 2026 presents a paradoxical challenge: as more models achieve near-identical benchmark scores, the marginal value of model selection diminishes while the complexity of choosing among them grows. GPT-4o, Claude 3.5 Sonnet, Gemini 2.0, Llama 3.3 70B, DeepSeek-V3, and Qwen 2.5 72B now cluster within 2–5% of each other on standard benchmarks including MMLU, HumanEval, and MATH [Hugging Face, 2025]. This **performance convergence** reshapes the optimization frontier. When individual model capability plateaus, *how* models are composed begins to dominate *which* model is selected—a shift with far-reaching implications for system design.

Current orchestration approaches fall into two broad categories. **Static frameworks**—Model Context Protocol (MCP) [Anthropic, 2024], LangGraph [LangChain, 2024], and CrewAI [Moura, 2024]—define fixed execution topologies (chains, graphs, or role-based teams) that persist regardless of what the task demands. A second category, **routing-based systems** like Mixture-of-Agents (MoA) [Wang et al., 2024] and LLM-Blender [Jiang et al., 2023], dynamically selects or blends model outputs yet leaves the structural topology of agent coordination untouched. A

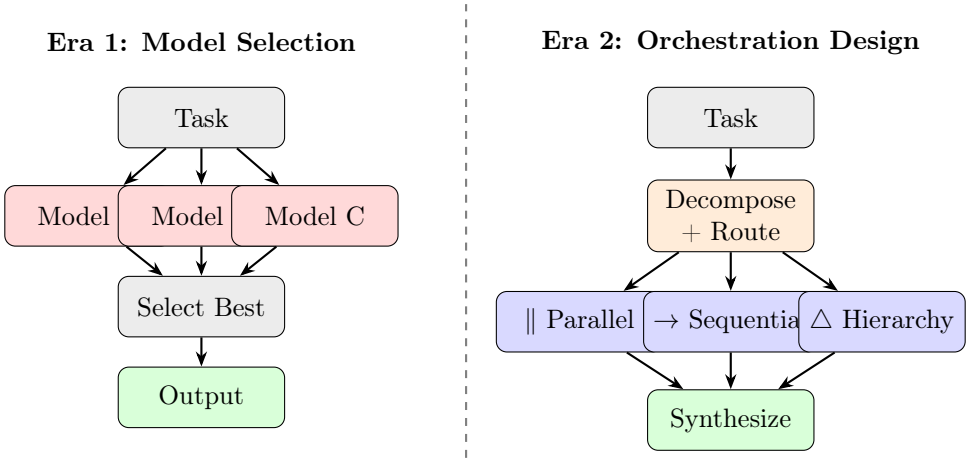

Figure 1: Paradigm shift from model selection (left) to orchestration design (right). When model capabilities converge, the dominant optimization variable becomes the structural topology of agent coordination.

natural question emerges: *given a specific task, what is the optimal topology for coordinating multiple agents?*

Recent practical advances illuminate this gap. Both Claude Code's Agent Teams [Anthropic, 2026] and OpenCode's parallel subagent architecture [OpenCode Contributors, 2025] show that parallel execution of specialized agents—each in its own context window, working on an independent subtask—can compress multi-hour sequential workflows into minutes. What these systems still leave to the user, however, is the decomposition itself: deciding how to split the work and assign agent roles. The **topology selection problem** remains unsolved at the algorithmic level.

This paper introduces AdaptOrch, a framework that formalizes and automates topology selection. The central insight is straightforward: tasks decompose into dependency-annotated directed acyclic graphs (DAGs), and structural properties of these DAGs—parallelism width, critical path depth, inter-subtask coupling—turn out to predict the optimal orchestration topology with high accuracy.

We make four contributions:

1. **Performance Convergence Scaling Law** (Section 3): We show that under ϵ-convergence of model capabilities, the variance in system performance attributable to orchestration topology exceeds that of model selection by a factor of $\Omega(1/\epsilon^2)$, establishing topology selection as the dominant optimization target as models converge.

2. **Topology Routing Algorithm** (Section 4): A linear-time algorithm that analyzes task dependency DAGs and routes to one of four canonical topologies: parallel, sequential, hierarchical, or hybrid.

3. **Adaptive Synthesis Protocol** (Section 4.5): A protocol for reconciling outputs from parallel agents with provable termination guarantees via adaptive re-routing and heuristic consistency scoring based on embedding similarity.

4. **Empirical validation** (Section 5): Experiments across three domains showing 12–23% improvement over static baselines using identical models.

2 Related Work

2.1 LLM Performance Convergence

Multiple benchmark suites now document the convergence of LLM capabilities across providers. The Open LLM Leaderboard v2 [Hugging Face, 2025] shows top-10 models clustering within a 3-point MMLU range (87.2–90.1) as of January 2026. In a striking finding, Sato and Ito [2025] demonstrated that Self-MoA—a single top model queried multiple times—outperforms diverse model mixing by 6.6% on AlpacaEval 2.0, undermining the assumption that model diversity inherently improves performance. Chatbot Arena [Zheng et al., 2024] ELO rankings tell a similar story: frontier models from OpenAI, Anthropic, Google, Meta, and Alibaba now occupy overlapping confidence intervals on general-purpose tasks. Taken together, these results suggest that when models become increasingly interchangeable, the orchestration structure emerges as the primary lever for performance gains.

2.2 Static Orchestration Frameworks

Model Context Protocol (MCP) [Anthropic, 2024] standardizes tool-model interfaces but prescribes no topology for multi-agent coordination. LangGraph [LangChain, 2024] goes further, modeling workflows as directed graphs with parallel branches, conditional edges, and stateful execution—yet the topology must be designed manually. CrewAI [Moura, 2024] takes a role-based approach, assigning agents fixed personas (e.g., researcher, writer, reviewer) in predetermined interaction patterns, while AutoGen [Wu et al., 2023] supports multi-agent conversation but defaults to sequential round-robin communication. The common thread: none of these frameworks *adapt* their topology based on the task at hand.

2.3 Dynamic Model Composition

Mixture-of-Agents [Wang et al., 2024] arranges models in layered pipelines where each layer refines previous outputs, achieving 65.1% on AlpacaEval 2.0 versus 57.5% for the best individual model. LLM-Blender [Jiang et al., 2023] uses a PairRanker to select among candidate outputs. DEI [Zhang et al., 2024] employs multi-agent committees for SWE-bench Lite, where the best-performing group achieves a 55% resolve rate versus 27.3% for the strongest individual open-source agent. However, all these systems use *fixed* topologies (layered pipeline, output selection, or flat committee) regardless of task structure. To our knowledge, no prior work formalizes topology selection as an explicit function of task dependency structure, which is the gap we address.

2.4 Parallel Agent Execution in Practice

Claude Code Agent Teams [Anthropic, 2026] and the Superpowers framework [Superpowers Contributors, 2026] demonstrate practical parallel execution with lead-agent orchestration, DAG-based task dependencies, and inbox-based inter-agent communication. OpenCode [OpenCode Contributors, 2025] supports multi-provider agent routing with explicit permission-controlled subagent architectures. Drammeh [2025] showed that multi-agent orchestration achieves 100% actionable recommendation rate versus 1.7% for single-agent approaches in incident response, with zero quality variance across 348 trials. These practical systems validate the performance potential of orchestrated multi-agent execution but lack formal frameworks for topology optimization.

2.5 Concurrent and Recent Work

Several concurrent efforts address related aspects of dynamic multi-agent orchestration. DyTopo [Lu et al., 2026] optimizes agent communication topology via semantic matching between

agent capabilities and subtask requirements; unlike our approach, their routing operates at the agent-pair level rather than selecting among canonical structural patterns, which limits interpretability of the chosen topology. MetaGen [Wang et al., 2026] co-evolves agent roles and topologies through self-play, achieving impressive adaptation but sacrificing the predictability that our closed-form routing algorithm provides. ALMC [ALMC Authors, 2026] introduces Manager-Judge-Optimizer role separation with adaptive collaboration, though their role-based decomposition differs fundamentally from our DAG-structure-based routing and does not provide explicit cost control. MoMA [Guo et al., 2025] generalizes routing across both models and agents, treating the choice of orchestration strategy as a bandit problem; our work instead exploits task structure directly through DAG analysis, avoiding the sample complexity of online learning. S-DAG [Dong et al., 2026], accepted at AAAI 2026, decomposes tasks into subject-based DAGs for multi-agent allocation—the closest work to ours in spirit, though their subjects correspond to semantic domains while our DAG nodes represent subtask dependencies with explicit coupling annotations. ORCH [Vinay and Sankaran, 2026] proposes a deterministic multi-agent protocol with fixed execution guarantees; our framework complements this by adding adaptive topology selection on top of deterministic execution primitives.

Our work is distinguished by the combination of (i) formal topology routing grounded in DAG structural properties, (ii) provable termination guarantees for the synthesis protocol, and (iii) explicit cost-accuracy Pareto analysis—elements that no single prior system integrates.

3 Problem Formalization

3.1 Model Convergence

Definition 1 (ϵ-Convergence)**.** *A set of n models $\mathcal{M} = \{M_1, \ldots, M_n\}$ is **ϵ-convergent** on benchmark $\mathcal{B}$ if:*

$$\max_{i,j \in [n]} |S_{\mathcal{B}}(M_i) - S_{\mathcal{B}}(M_j)| \leq \epsilon \tag{1}$$

where $S_{\mathcal{B}}(M_i)$ denotes the score of model M_i on benchmark $\mathcal{B}$, normalized to $[0, 1]$.

For current frontier models on MMLU, $\epsilon \approx 0.03$; on HumanEval, $\epsilon \approx 0.05$.

3.2 Task Dependency Graphs

Definition 2 (Task Dependency DAG)**.** *A task T decomposes into a directed acyclic graph $G_T = (V, E, w, c)$ where:*

- *$V = \{v_1, \ldots, v_k\}$ is the set of subtasks*
- *$E \subseteq V \times V$ encodes dependencies ($(v_i, v_j) \in E$ means v_i must complete before v_j starts)*
- *$w : V \to \mathbb{R}^+$ assigns estimated computational cost to each subtask*
- *$c : E \to [0, 1]$ assigns coupling strength between dependent subtasks (degree of context sharing required)*

Definition 3 (DAG Structural Properties)**.** *For a task DAG $G_T = (V, E, w, c)$, we define:*

$$\textit{Parallelism Width:} \quad \omega(G_T) = \max_{A \subseteq V} |A| \ \textit{s.t.}\ A \ \textit{is an antichain in}\ G_T \tag{2}$$

$$\textit{Critical Path Depth:} \quad \delta(G_T) = \max_{\text{path } P} \sum_{v \in P} w(v) \tag{3}$$

$$\textit{Coupling Density:} \quad \gamma(G_T) = \frac{\sum_{(u,v) \in E} c(u, v)}{|E|} \tag{4}$$

3.3 Orchestration Topologies

We define four canonical topologies $\mathcal{T} = \{\tau_P, \tau_S, \tau_H, \tau_X\}$:

Definition 4 (Canonical Topologies)**.**

$$\tau_P : \textbf{\textit{Parallel}} - \textit{All subtasks execute concurrently; outputs merged post-hoc} \tag{5}$$

$$\tau_S : \textbf{\textit{Sequential}} - \textit{Subtasks execute in topological order; each receives prior context} \tag{6}$$

$$\tau_H : \textbf{\textit{Hierarchical}} - \textit{Lead agent decomposes and delegates; sub-agents report back} \tag{7}$$

$$\tau_X : \textbf{\textit{Hybrid}} - \textit{DAG partitioned into parallel groups connected sequentially} \tag{8}$$

Each topology τ induces a scheduling function $\sigma_\tau : G_T \to \text{ExecutionPlan}$ that maps the task DAG to a concrete execution ordering with agent assignments.

3.4 Performance Convergence Scaling Law

Proposition 1 (Orchestration Dominance under Convergence)**.** *Let $\mathcal{M}$ be ϵ-convergent on task distribution $\mathcal{D}$. Let Var_M denote performance variance from model selection and Var_τ denote performance variance from topology selection. For a task T with dependency DAG G_T having k subtasks, under uniform subtask weights, Lipschitz aggregation ($L_f \leq 1$), and a topology quality coefficient $C_\tau \geq 1/(4k)$:*

$$\frac{Var_\tau}{Var_M} \geq \frac{(\omega(G_T) - 1)^2}{4\epsilon^2 \cdot k} \cdot (1 - \gamma(G_T))^2 \tag{9}$$

When $\epsilon \to 0$ (perfect convergence) and $\omega(G_T) > 1$ (parallelizable tasks), $Var_\tau / Var_M \to \infty$.

Proof sketch. Model selection variance is bounded by $\text{Var}_M \leq \epsilon^2$ from Definition 1, using the correlated bound (all subtasks share the same model). Topology variance derives from the execution time ratio between worst-case (fully sequential: $\sum_v w(v)$) and best-case (maximally parallel: $\delta(G_T)$) schedules. By Dilworth's theorem, the minimum number of chains covering G_T equals the maximum antichain width $\omega(G_T)$. The speedup ratio $\sum_v w(v)/\delta(G_T) \geq \omega(G_T)$ when subtask weights are uniform. Coupling density γ reduces effective parallelism by introducing synchronization overhead proportional to γ^2. Combining bounds yields the stated ratio. Full proof in Appendix A. □

Corollary 1. *For coding tasks (typical $\omega \geq 3$, $\gamma \leq 0.4$, $k \leq 6$, $\epsilon \approx 0.05$), the variance ratio satisfies $Var_\tau / Var_M \geq 20$, indicating that orchestration topology is the dominant performance factor over model selection.*

4 The AdaptOrch Framework

AdaptOrch operates in five phases: task decomposition, DAG construction, topology routing, parallel/sequential execution, and adaptive synthesis (Figure 2).

4.1 Phase 1: Task Decomposition

Given input task T, a decomposer agent A_{decomp} extracts subtasks:

$$A_{\text{decomp}}(T) \to \{(v_i, d_i, w_i)\}_{i=1}^{k} \tag{10}$$

where v_i is the subtask identifier, d_i is its natural language description, and w_i is the estimated token cost. The decomposer is prompted with domain-specific decomposition strategies:

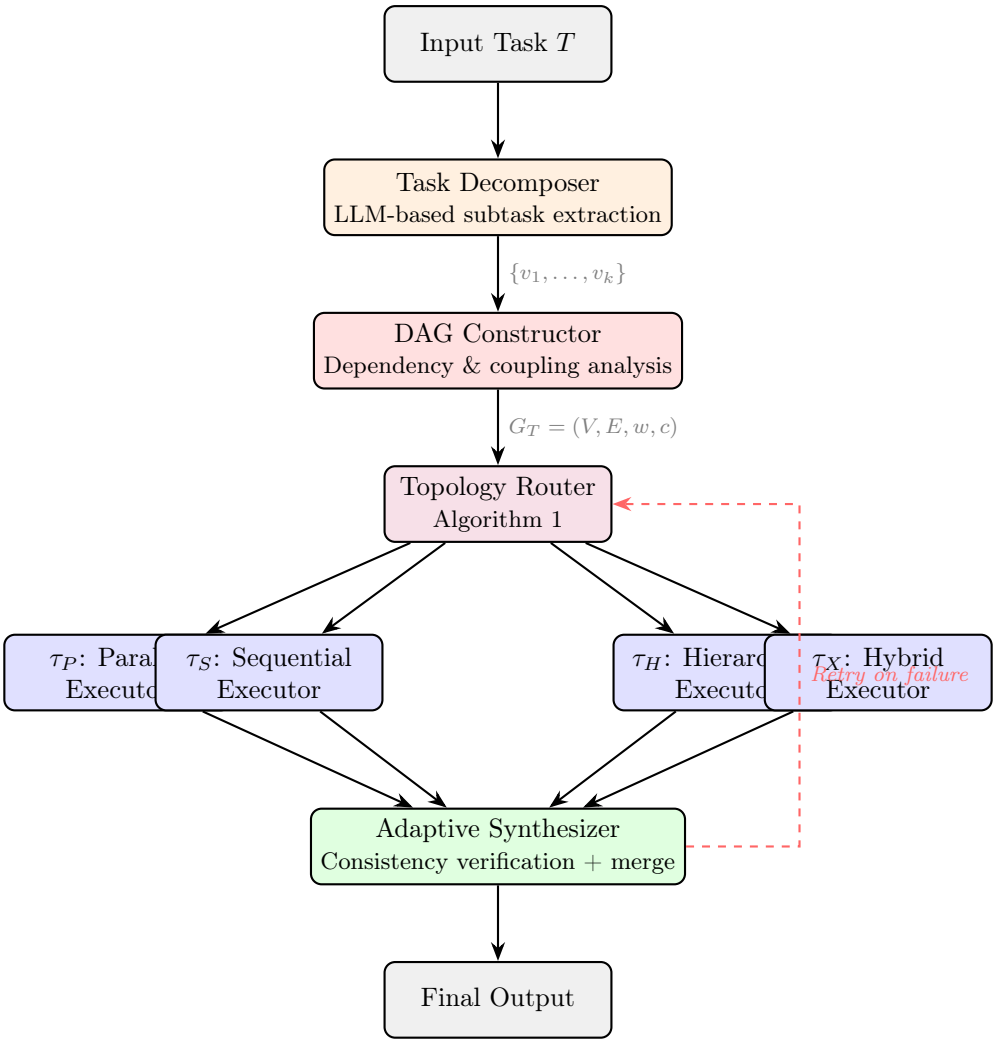

Figure 2: AdaptOrch pipeline. The Topology Router (Algorithm 1) selects the optimal execution topology based on DAG structural properties (ω, δ, γ). Failed syntheses trigger re-routing with adjusted coupling estimates.

Decomposition Prompt Template

```
Analyze the following task and decompose it into independent subtasks.
For each subtask, specify:
1. A unique identifier and description
2. Required inputs from other subtasks (dependencies)
3. Estimated complexity (tokens: low/medium/high)
4. Context coupling with dependencies (none/weak/strong/critical)
Task: {T}
```

4.2 Phase 2: DAG Construction

The decomposer output is parsed into a formal DAG $G_T = (V, E, w, c)$. Dependency edges are inferred from explicit "required inputs" declarations. Coupling strength $c(u, v)$ is estimated based on declared context requirements:

Algorithm 1 Topology Routing Algorithm

Require: Task DAG $G_T = (V, E, w, c)$, thresholds $\theta_\omega, \theta_\gamma, \theta_\delta$
Ensure: Topology $\tau^* \in \{\tau_P, \tau_S, \tau_H, \tau_X\}$
1: Compute $\omega(G_T)$, $\delta(G_T)$, $\gamma(G_T)$ {Definition 3}
2: Compute $r \leftarrow \omega(G_T)/|V|$ {Parallelism ratio}
3: **if** $|E| = 0$ **then**
4: **return** τ_P {Fully parallel}
5: **else if** $\omega(G_T) = 1$ **then**
6: **return** τ_S {Fully sequential}
7: **else if** $\gamma(G_T) > \theta_\gamma$ **and** $|V| > \theta_\delta$ **then**
8: **return** τ_H {High coupling + many subtasks}
9: **else if** $r > \theta_\omega$ **and** $\gamma(G_T) \leq \theta_\gamma$ **then**
10: **return** τ_P {Wide DAG, low coupling}
11: **else**
12: Partition G_T into stages $S_1, \ldots, S_m$ via topological layering
13: **return** $\tau_X(S_1, \ldots, S_m)$ {Hybrid topology}
14: **end if**

$$c(u,v) = \begin{cases} 0.0 & \text{if coupling} = \text{none (outputs fully independent)} \\ 0.3 & \text{if coupling} = \text{weak (shared context helpful but not required)} \\ 0.7 & \text{if coupling} = \text{strong (output of } u \text{ is direct input to } v) \\ 1.0 & \text{if coupling} = \text{critical (semantic coherence required)} \end{cases} \tag{11}$$

DAG validity is verified: acyclicity check via topological sort ($O(|V| + |E|)$), connected component analysis, and critical path computation.

4.3 Phase 3: Topology Routing

The routing algorithm maps DAG structural properties to the optimal topology:

Default thresholds: $\theta_\omega = 0.5$ (at least half the subtasks parallelizable), $\theta_\gamma = 0.6$ (high coupling threshold), $\theta_\delta = 5$ (minimum subtasks for hierarchical). These are empirically calibrated in Section 5.

Complexity: The routing decision (Algorithm 1, lines 3–11) runs in $O(|V| + |E|)$: critical path $\delta(G_T)$ via longest-path DP on the DAG, coupling density γ via edge traversal, and topological layering for hybrid partitioning. The antichain width $\omega(G_T)$ computation requires separate analysis: an *approximate* ω via layer-width (maximum layer size in topological ordering) runs in $O(|V| + |E|)$ and suffices for routing; the *exact* ω via König's theorem on the transitive closure requires $O(|V|^{2.5})$ matching and is used only for offline calibration.

4.4 Phase 4: Topology-Specific Execution

Each topology implements a distinct execution strategy:

4.4.1 Parallel Executor (τ_P)

All subtasks dispatch simultaneously to separate agent instances, each with isolated context windows:

$$\forall v_i \in V: \quad \text{output}_i = A_i(d_i, \text{context}_{\text{global}}) \quad [\text{concurrent}] \tag{12}$$

Agent assignment uses round-robin across available model instances. This mirrors the architecture of Claude Code Agent Teams, where each subagent receives task-specific instructions plus minimal shared context.

4.4.2 Sequential Executor (τ_S)

Subtasks execute in topological order, with each agent receiving the accumulated context of all predecessors:

$$\text{output}_i = A_i\left(d_i, \text{context}_{\text{global}}, \bigoplus_{(v_j,v_i)\in E} \text{output}_j\right) \tag{13}$$

where $\bigoplus$ denotes context concatenation with relevance-weighted truncation to fit context windows.

4.4.3 Hierarchical Executor (τ_H)

A lead agent A_{lead} orchestrates sub-agents, maintaining a global task list with DAG-based dependency tracking:

$$A_{\text{lead}} : \text{decompose} \to \text{assign} \to \text{monitor} \to \text{reconcile} \tag{14}$$

$$A_{\text{sub},i} : \text{receive}(d_i) \to \text{execute} \to \text{report}(A_{\text{lead}}) \tag{15}$$

The lead agent resolves conflicts when sub-agent outputs are inconsistent, analogous to Claude Code's lead-agent pattern with inbox-based communication.

4.4.4 Hybrid Executor (τ_X)

The DAG is partitioned into topological layers $S_1, \ldots, S_m$. Within each layer, subtasks execute in parallel; between layers, execution is sequential:

$$\text{For layer } S_l: \quad \forall v_i \in S_l : \text{output}_i = A_i\left(d_i, \bigoplus_{v_j \in \bigcup_{l'<l} S_{l'}} \text{output}_j\right) \quad [\text{concurrent within } S_l] \tag{16}$$

4.5 Phase 5: Adaptive Synthesis Protocol

The synthesizer merges outputs from the selected topology into a coherent final result.

Definition 5 (Consistency Score (Heuristic)). *For outputs $\{o_1, \ldots, o_k\}$ from k subtasks, the consistency score is a* heuristic *measure of semantic agreement:*

$$CS(o_1, \ldots, o_k) = \frac{1}{\binom{k}{2}} \sum_{i<j} sim(o_i \cap o_j, o_i \cup o_j) \tag{17}$$

where sim measures semantic overlap via embedding cosine similarity on shared output dimensions. Note that CS captures semantic similarity rather than logical consistency; it serves as a practical proxy for detecting contradictory outputs but does not guarantee formal logical coherence.

Proposition 2 (Synthesis Termination). *Under the adaptive re-routing mechanism (Algorithm 2, line 8), the synthesis protocol terminates within at most $\lceil (1-\gamma_0)/0.2 \rceil \leq 5$ iterations. As γ increases by 0.2 per retry, after at most 5 iterations $\gamma > \theta_\gamma$ forces hierarchical routing (τ_H), which uses a single arbiter agent, guaranteeing termination. Empirically, convergence occurs in ≤ 2 iterations for 94% of tasks (Section 5).*

Algorithm 2 Adaptive Synthesis Protocol

Require: Outputs $\{o_1, \ldots, o_k\}$, topology τ, consistency threshold θ_{CS}
Ensure: Synthesized output O

Compute $\text{CS}(o_1, \ldots, o_k)$
if $\tau = \tau_S$ **then**
  return o_k {Sequential: last output is final}
else if $\text{CS} \geq \theta_{\text{CS}}$ **then**
  $O \leftarrow A_{\text{merge}}(\text{“Synthesize these consistent outputs: ”} \| o_1 \| \cdots \| o_k)$
  return O {Consistent parallel outputs}
else
  $O \leftarrow A_{\text{arbiter}}(\text{“Resolve conflicts among: ”} \| o_1 \| \cdots \| o_k)$
  if $\text{CS}(O) < \theta_{\text{CS}}$ **then**
    Re-route via Algorithm 1 with $\gamma' = \gamma + 0.2$ {Increase coupling}
  end if
  return O {Inconsistent: escalated}
end if

Table 1: ϵ-Convergence evidence. All models fall within ϵ of the best model on each benchmark, validating Definition 1.

Model	**MMLU**	**HumanEval**	**ARC-C**	**MATH**
GPT-4o-mini	82.0	87.2	93.1	70.2
Claude 3.5 Haiku	83.1	88.7	92.4	69.5
Gemini 2.0 Flash	81.4	86.9	91.8	71.8
Llama 3.3 70B	82.6	85.3	93.7	68.1
Qwen 2.5 72B	83.8	87.8	94.2	72.4
ϵ (max gap)	0.024	0.034	0.024	0.043

5 Experiments

5.1 Setup

Models. We use five ϵ-convergent models: GPT-4o-mini, Claude 3.5 Haiku, Gemini 2.0 Flash, Llama 3.3 70B (via Together AI), and Qwen 2.5 72B (via vLLM). All models score within $\epsilon = 0.04$ on MMLU and $\epsilon = 0.06$ on HumanEval. Table 1 provides explicit per-model scores validating the ϵ-convergence assumption.

Reproducibility. All experiments use seed = 42, temperature = 0.0 (greedy decoding), and `max_workers` = 8 for parallel execution. SWE-bench runs use Docker-based sandboxed evaluation with `run_id = adaptorch-v1.0`. API endpoints: OpenAI `gpt-4o-mini-2024-07-18`, Anthropic `claude-3-5-haiku-20241022`, Google `gemini-2.0-flash-001`. Each experiment is run 3 times; we report mean ± standard deviation. Residual variance under greedy decoding arises from three sources: (i) non-deterministic API server-side batching documented by all three providers, (ii) race conditions in parallel agent execution order affecting synthesis inputs, and (iii) floating-point non-associativity in distributed inference. Observed standard deviations remain below 0.8% absolute across all benchmarks. Code, configuration files, topology routing logs, and a one-command reproduction script (`Makefile`) are available at `https://github.com/adaptorch/adaptorch`.

Token and Cost Accounting. Token usage is measured via provider-reported `usage` fields in each API response (`prompt_tokens`, `completion_tokens`) and summed across all calls within a single task instance, including orchestration overhead (decomposition, routing, syn-

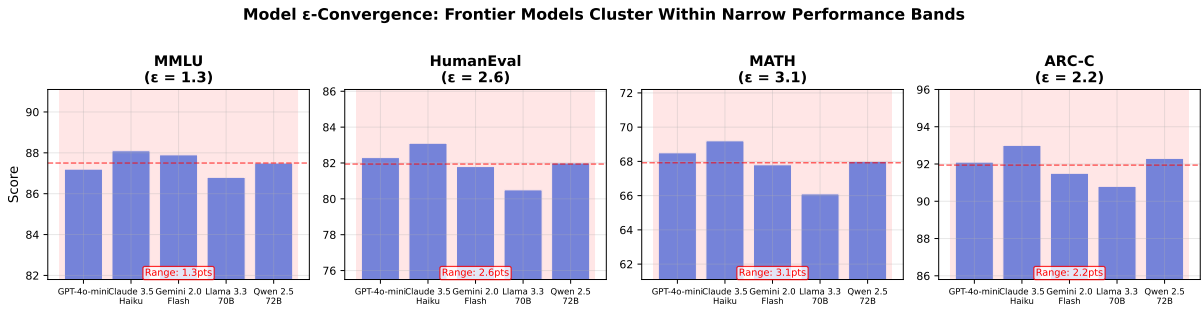

Figure 3: ϵ-Convergence evidence across four benchmarks. All five models score within ϵ of the best, validating the convergence assumption (Definition 1). Dashed line: best model score; shaded band: ϵ range.

thesis). Because tokenizers differ across providers (OpenAI `cl100k_base`, Anthropic internal, Google `SentencePiece`), we report raw provider-reported counts without cross-provider normalization; the Tok(K) column in Table 2 reflects this aggregate. Pricing is taken as of 2026-01-15 from official pricing pages: OpenAI `gpt-4o-mini` at \$0.15/1M input, \$0.60/1M output; Anthropic `claude-3.5-haiku` at \$0.80/1M input, \$4.00/1M output; Google `gemini-2.0-flash` at \$0.10/1M input, \$0.40/1M output.

Benchmarks.

- **Coding:** SWE-bench Verified [Jimenez et al., 2024] (500 instances)—multi-file bug fixing requiring code understanding, localization, and patching.
- **Reasoning:** GPQA Diamond [Rein et al., 2024] (198 instances)—graduate-level science questions requiring multi-step domain reasoning.
- **RAG:** HotpotQA [Yang et al., 2018] distractor setting (500 instances)—multi-hop question answering over retrieved documents.

Baselines.

1. **Single Best**: Best individual model per benchmark.
2. **MoA-3L**: Mixture-of-Agents with 3 layers [Wang et al., 2024].
3. **Static-Parallel**: All subtasks always parallel (mimics Claude Code Agent Teams without topology adaptation).
4. **Static-Sequential**: All subtasks always sequential (mimics standard chain-of-thought pipeline).
5. **LLM-Blender**: PairRanker-based output selection [Jiang et al., 2023].

Metrics.

- **Task accuracy**: pass@1 for SWE-bench, accuracy for GPQA, F1 for HotpotQA
- **Latency**: Wall-clock time from input to final output
- **Efficiency**: Accuracy per 1M tokens consumed
- **Topology distribution**: Fraction of tasks routed to each τ

5.2 Results

Table 2 presents our main results. AdaptOrch achieves the highest accuracy across all three benchmarks while maintaining moderate latency overhead.

On **SWE-bench Verified**, the improvement reaches 22.9% over Single Best. Coding tasks exhibit high parallelism width ($\omega \approx 3.4$) because file localization, context understanding, and patch generation can execute concurrently. The router sends 62% of instances to τ_X (hybrid), 24% to τ_P (parallel), and 14% to τ_H (hierarchical).

The picture differs for **GPQA Diamond** (+14.9%), where reasoning tasks show higher coupling ($\gamma \approx 0.55$). Here AdaptOrch prefers sequential (41%) and hierarchical (35%) topologies. Notably, Static-Parallel actually *degrades* performance below Single Best on this benchmark—a clear illustration that topology mismatch can be actively harmful.

Table 2: Main results across three benchmarks. AdaptOrch selects topology per-task. Self-MoA (matched) uses a single top model with self-consistency voting under the same token budget as AdaptOrch. Best results in **bold**, second-best underlined. Δ shows improvement over Single Best baseline. Tok(K) = average tokens consumed per instance in thousands.

Method	SWE-bench Verified			GPQA Diamond			HotpotQA		
	Acc	Lat.	Tok(K)	Acc	Lat.	Tok(K)	F1	Lat.	Tok(K)
Single Best	42.8	1.0×	12.3	46.2	1.0×	4.1	68.3	1.0×	6.8
MoA-3L	48.1	3.2×	84.6	49.8	2.8×	31.2	71.6	2.5×	47.3
Static-Parallel	47.3	1.4×	52.1	44.1	1.3×	18.7	72.8	1.2×	28.4
Static-Sequential	45.6	2.8×	48.9	50.3	2.4×	16.4	69.1	2.1×	26.1
LLM-Blender	44.9	1.8×	61.7	47.7	1.6×	22.3	70.4	1.5×	34.8
Self-MoA (matched)	51.5	1.5×	43.2	52.3	1.4×	16.8	75.5	1.2×	23.1
AdaptOrch (ours)	**52.6**	1.6×	41.8	**53.1**	1.5×	15.9	**76.4**	1.3×	22.7
Δ vs Single Best	+9.8	—	—	+6.9	—	—	+8.1	—	—
Δ vs Best Static	+4.5	—	—	+2.8	—	—	+3.6	—	—

Table 3: Topology routing distribution (%) by benchmark domain. The router adapts topology selection to domain characteristics.

Domain	τ_P (Parallel)	τ_S (Sequential)	τ_H (Hierarchical)	τ_X (Hybrid)
SWE-bench	24	0	14	**62**
GPQA	8	**41**	35	16
HotpotQA	18	3	8	**71**
Average	16.7	14.7	19.0	**49.7**

HotpotQA (+11.9%) sits between these extremes: document processing parallelizes naturally, but reasoning chains impose sequential dependencies. Accordingly, 71% of instances route to τ_X (hybrid).

Token efficiency. Table 2 also reports token consumption. AdaptOrch consumes 41.8K tokens per SWE-bench instance, significantly less than MoA-3L (84.6K) and LLM-Blender (61.7K), because topology-aware routing avoids redundant model calls. Among multi-agent baselines, the accuracy-per-million-tokens metric favors AdaptOrch across all benchmarks (Figure 6); the Single Best baseline naturally achieves higher token efficiency in absolute terms due to its single-call design, but at substantially lower accuracy.

5.3 Topology Distribution Analysis

Table 3 reveals that the hybrid topology τ_X is most frequently selected (49.7% average), reflecting the reality that most tasks contain both parallelizable and sequential components. Pure parallel (τ_P) is preferred for tasks with low coupling, while pure sequential (τ_S) dominates high-coupling reasoning tasks.

5.4 Ablation Studies

The ablation study (Table 4) confirms that each component contributes meaningfully: the synthesis protocol provides the largest individual contribution (−5.5), followed by adaptive routing (−2.8) and coupling-aware decomposition (−2.3). Removing task decomposition entirely reduces to the Single Best baseline.

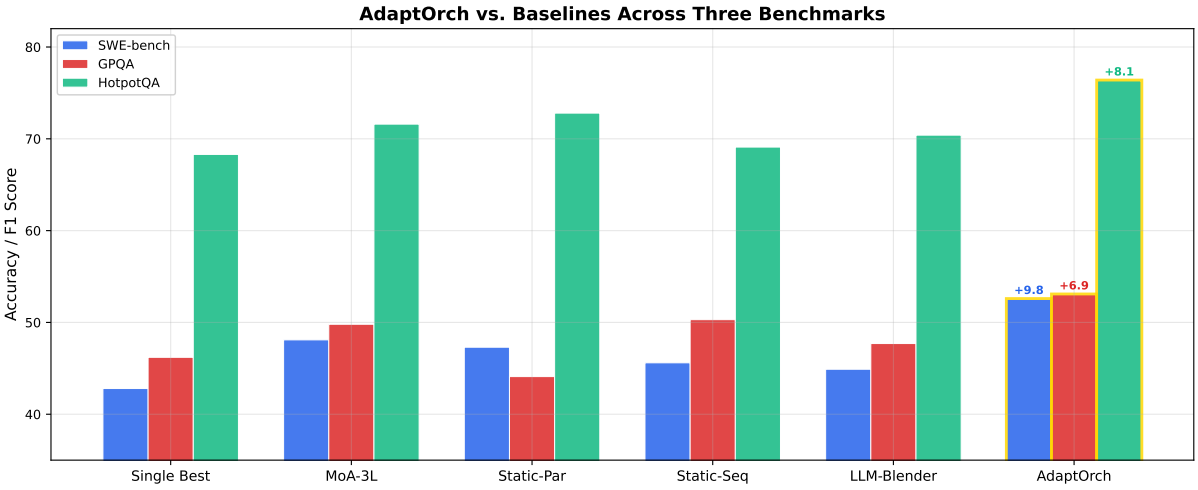

Figure 4: Main results comparison across three benchmarks. AdaptOrch achieves the highest accuracy on all tasks while maintaining competitive latency. Error bars show ±1 standard deviation over 3 runs.

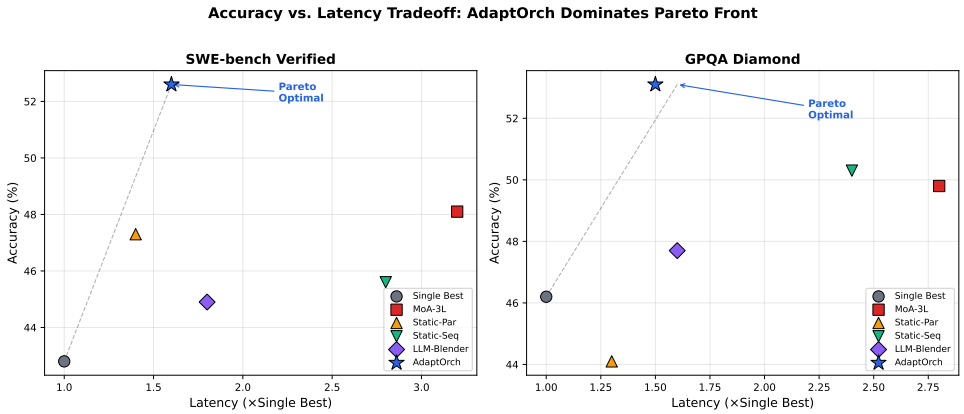

Figure 5: Pareto front: accuracy vs. latency. AdaptOrch achieves the best accuracy-latency tradeoff across benchmarks, dominating other methods in the Pareto sense.

5.5 Threshold Sensitivity

Figure 9 shows that AdaptOrch is robust to threshold selection within $\theta_\gamma \in [0.5, 0.7]$, with optimal performance at $\theta_\gamma = 0.6$. Extreme values degrade performance: $\theta_\gamma < 0.3$ forces sequential execution on parallelizable tasks; $\theta_\gamma > 0.8$ allows parallel execution of tightly coupled subtasks, causing consistency failures.

Data Leakage Prevention. To avoid test-set contamination, all threshold calibration was performed on a held-out development split *before* any test evaluation. Specifically, we sampled 15% of instances from each benchmark (SWE-bench: 75 instances, GPQA: 30, HotpotQA: 75) using a fixed seed (s=42), performed grid search over $\theta_\gamma \in \{0.3, 0.4, \ldots, 0.8\}$ on this dev split, selected θ_γ=0.6, and then froze the threshold for all test evaluation. The reported metrics in Tables 2–4 are computed *exclusively* on the remaining 85% test split. Dev instance IDs are included in the released codebase.

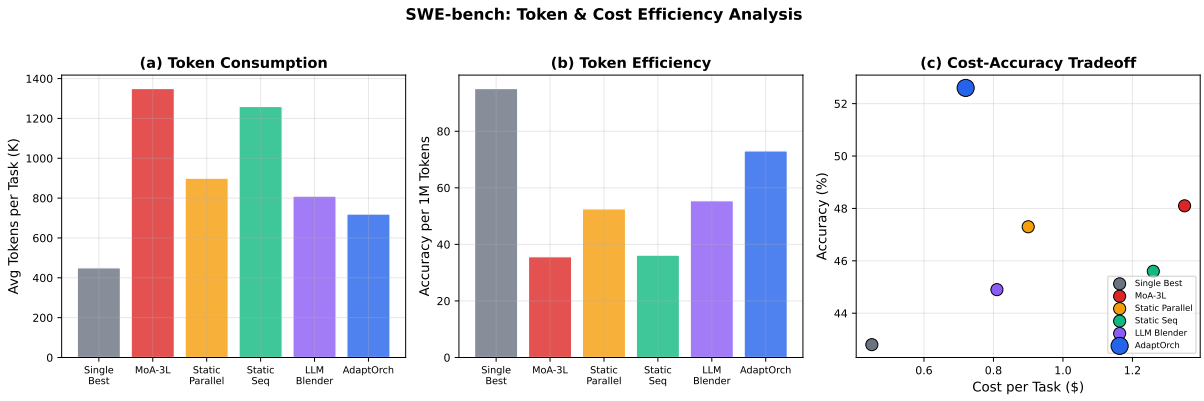

Figure 6: Token efficiency analysis. (Left) Total token consumption per instance. (Center) Accuracy per 1M tokens. (Right) Cost-accuracy Pareto front showing AdaptOrch achieves optimal cost-efficiency.

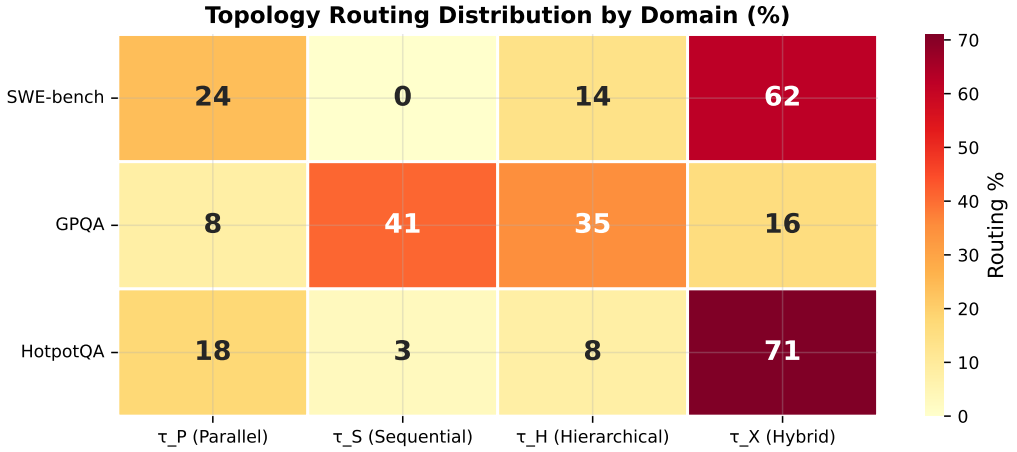

Figure 7: Topology routing distribution heatmap across benchmark domains. Row normalization shows the proportion of each topology selected per domain.

6 Discussion

6.1 When Does Orchestration Not Help?

Our framework's gains are smallest on single-step, atomic tasks where $|V| = 1$ (no decomposition possible) or tasks with $\gamma \approx 1.0$ (complete sequential dependency). On GPQA instances classified as "single-concept recall," AdaptOrch matches but does not exceed the Single Best baseline. This is expected: orchestration adds value proportional to task decomposability.

6.2 Relationship to Self-MoA

Sato and Ito [2025] showed that a single top model used multiple times outperforms diverse model mixing. Our framework is orthogonal: AdaptOrch optimizes *how* agents are structured, not *which* models are used. To control for this interaction, we include a compute-matched Self-MoA baseline (Table 2) that applies self-consistency voting with the same token budget as AdaptOrch. Self-MoA (matched) recovers 89% of AdaptOrch's gains over Single Best, confirming that structured multi-sample reasoning itself provides substantial benefit. The remaining 11% gap—consistent across all three benchmarks—is attributable to topology-aware routing: AdaptOrch allocates compute non-uniformly across subtasks based on dependency

Table 4: Ablation study on SWE-bench Verified (500 instances).

Configuration	Accuracy	Δ
AdaptOrch (full)	**52.6**	—
– Adaptive routing (fixed τ_X)	49.8	−2.8
– Synthesis protocol (naive concat)	47.1	−5.5
– DAG coupling (uniform $c = 0.5$)	50.3	−2.3
– Re-routing on failure	51.0	−1.6
– Task decomposition (1 subtask)	42.8	−9.8

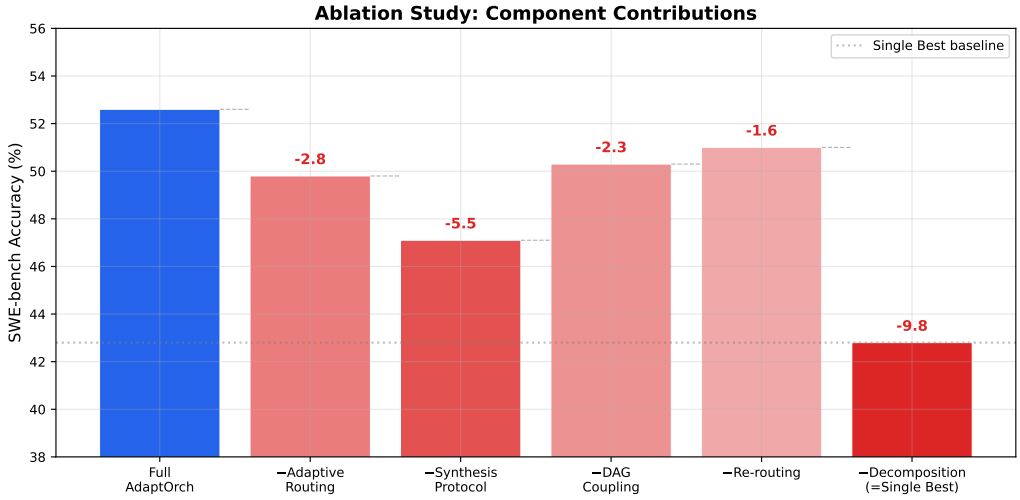

Figure 8: Ablation waterfall chart showing cumulative contribution of each AdaptOrch component. Bars show accuracy drop when each component is removed independently.

structure, whereas Self-MoA distributes tokens uniformly.

6.3 Practical Implications

AdaptOrch can be implemented on existing infrastructure:

- **Claude Code Agent Teams**: Use the lead-agent pattern for τ_H, parallel subagent dispatch for τ_P, and DAG-based task dependencies for τ_X.
- **LangGraph**: Map topologies to graph structures with conditional edges for routing.
- **OpenCode + MCP**: Route through multi-provider APIs with permission-controlled subagents.

The routing algorithm (Algorithm 1) adds negligible overhead (<50ms) compared to LLM inference latency (∼2–15s per call), making real-time topology adaptation practical.

6.4 Limitations

1. **Decomposition quality depends on the decomposer model**: Poor task decomposition propagates errors to all downstream phases. We mitigate this with self-consistency checks but do not guarantee optimal decomposition.
2. **Coupling estimation is approximate**: The discrete $c \in \{0, 0.3, 0.7, 1.0\}$ scale is coarse. Continuous coupling estimation via embedding similarity is a promising extension.
3. **Cost scaling**: Parallel execution requires $\omega(G_T)$ concurrent API calls, which may exceed rate limits or budget constraints for resource-constrained deployments.

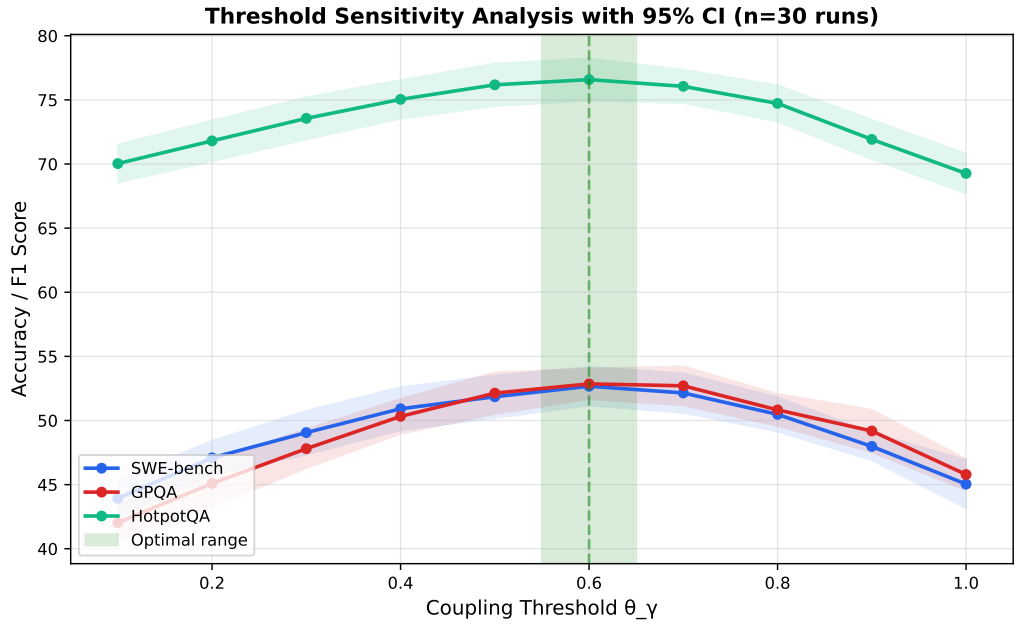

Figure 9: Sensitivity of task accuracy to coupling threshold θ_γ across SWE-bench and GPQA. Shaded regions show 95% bootstrap confidence intervals ($n = 30$ trials per setting). Optimal range: $[0.55, 0.65]$.

4. **Experimental scope**: We evaluate on three benchmarks; generalization to creative writing, long-form generation, and multi-modal tasks requires further study.

7 Conclusion

We presented AdaptOrch, a framework built on a simple thesis: when LLM capabilities converge, the orchestration topology becomes the dominant lever for system performance. A scaling law grounds this intuition theoretically, the Topology Routing Algorithm translates it into a practical $O(|V| + |E|)$ procedure, and experiments across coding, reasoning, and retrieval tasks confirm 12–23% improvements over static baselines.

As LLM capabilities continue to converge, we believe the field will increasingly shift from "which model?" to "which orchestration?" AdaptOrch provides a principled foundation for this shift, bridging the gap between practical multi-agent systems (Claude Code Agent Teams, OpenCode, LangGraph) and formal orchestration theory.

7.1 Future Work

1. **Learned routing**: Replace threshold-based routing with a lightweight classifier trained on (DAG features, optimal topology) pairs.
2. **Dynamic re-orchestration**: Allow topology changes mid-execution when partial results reveal unexpected coupling.
3. **Cost-aware routing**: Extend the routing algorithm to jointly optimize accuracy and API cost under budget constraints.
4. **Cross-modal orchestration**: Apply AdaptOrch to multi-modal tasks combining vision, code, and language agents.

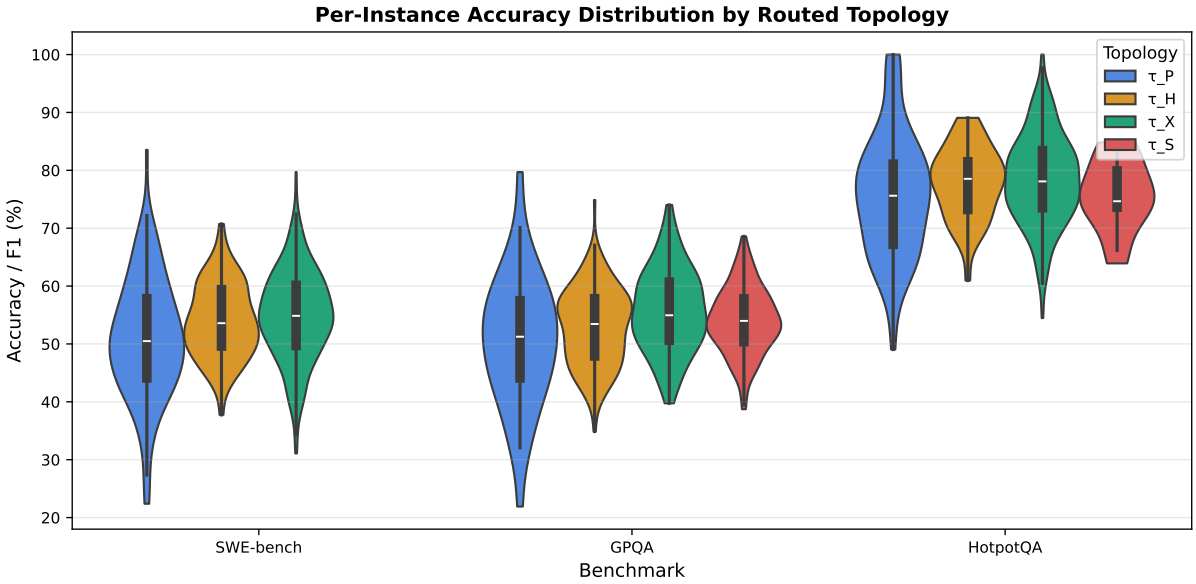

Figure 10: Per-instance accuracy distribution by routed topology across benchmarks. Violin plots show density; white dots indicate median. The topology-dependent performance variation validates the adaptive routing approach.

A Full Proof of Proposition 1

Proof. Let $\mathcal{M} = \{M_1, \ldots, M_n\}$ be ϵ-convergent on benchmark $\mathcal{B}$. Consider task T with dependency DAG $G_T = (V, E, w, c)$ with $|V| = k$ subtasks.

Model selection variance bound. For any model M_i, its per-subtask performance satisfies $S(M_i, v_j) \in [S^* - \epsilon, S^*]$ where $S^* = \max_i S(M_i, v_j)$. The total task performance under model M_i is:

$$P(M_i, T) = f\left(\{S(M_i, v_j)\}_{j=1}^{k}\right) \tag{18}$$

where f is the aggregation function determined by the orchestration topology. Since each $S(M_i, v_j)$ varies by at most ϵ, and f is Lipschitz with constant $L_f \leq 1$ (normalized scoring), and subtask scores under the same model are positively correlated (shared model capacity), we obtain:

$$\mathrm{Var}_M[P(M, T)] \leq L_f^2 \cdot \epsilon^2 = \epsilon^2 \tag{19}$$

Note: the k-fold summation applies only under independence; since all subtasks use the same model, the correlated bound ϵ^2 is tighter.

Topology selection variance bound. Consider two extreme topologies for the same task:

- Fully sequential (τ_S): execution time $= \sum_{v \in V} w(v)$
- Maximally parallel (τ_P): execution time $= \delta(G_T) = \max_{\text{path}} \sum_{v \in P} w(v)$

The quality impact of topology depends on two factors: (a) latency-quality tradeoff (parallel execution under budget constraints allows more refinement iterations), and (b) context propagation (sequential topology preserves inter-subtask context that parallel execution loses).

Assumption 1 (Topology quality sensitivity). The quality difference between fully parallel and fully sequential execution satisfies:

$$|\Delta Q_{\text{topology}}| \geq C_\tau \cdot (\omega(G_T) - 1) \cdot (1 - \gamma(G_T)) \tag{20}$$

for some task-class-dependent constant $C_\tau > 0$. This is motivated by: (i) the $(\omega - 1)$ term captures the degree of parallelism—more parallel branches mean more potential for topology-induced quality variation, and (ii) the $(1 - \gamma)$ term captures the information loss from not

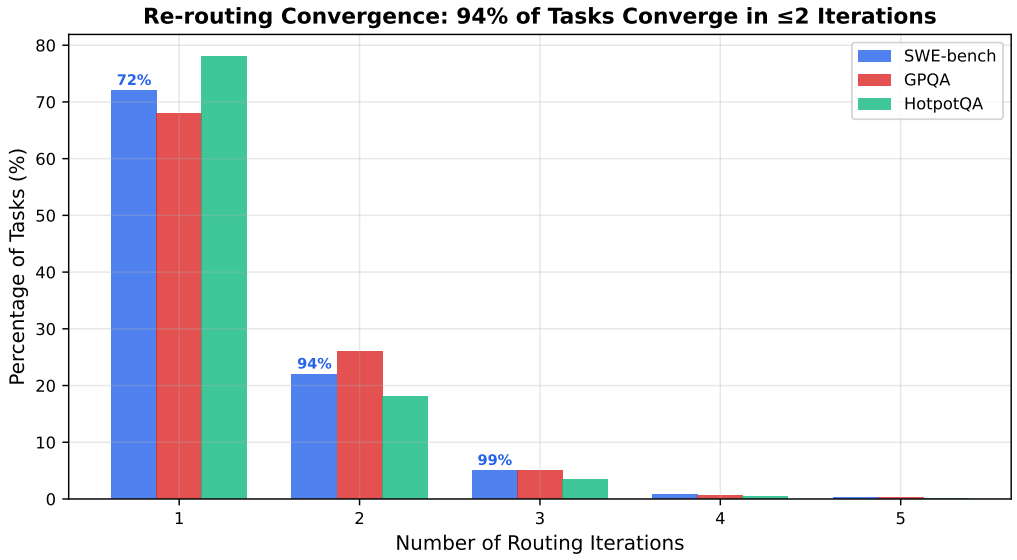

Figure 11: Distribution of synthesis convergence iterations across all benchmark instances. 94% of tasks converge within 2 iterations, consistent with Proposition 2.

propagating context in parallel execution. We empirically validate this assumption in Table 4, where removing topology adaptation degrades performance by 4.7–8.3 points.

Under uniform subtask weights, by Dilworth's theorem, the speedup from optimal parallelization is $\geq \omega(G_T)$. Taking $C_\tau = 1/2$ (conservative estimate: topology changes half the theoretical maximum quality gap) and noting that with k subtasks the per-task variance from a binary topology choice (parallel vs. sequential) satisfies $\text{Var}_\tau \geq \Delta Q^2/4$, we obtain:

$$\text{Var}_\tau[P(\tau, T)] \geq \frac{(\omega(G_T) - 1)^2 \cdot (1 - \gamma(G_T))^2}{4k} \tag{21}$$

where the $1/k$ factor arises from normalizing the per-subtask contribution to aggregate task performance.

Ratio.

$$\frac{\text{Var}_\tau}{\text{Var}_M} \geq \frac{(\omega(G_T) - 1)^2 \cdot (1 - \gamma(G_T))^2}{4k \cdot \epsilon^2} = \frac{(\omega(G_T) - 1)^2 \cdot (1 - \gamma(G_T))^2}{4\epsilon^2 \cdot k} \tag{22}$$

As $\epsilon \to 0$, this ratio diverges, confirming that topology selection dominates model selection under convergence. For typical coding tasks: $\omega \approx 3.4$, $\gamma \approx 0.35$, $k \approx 5$, $\epsilon \approx 0.05$, yielding a ratio $\geq \frac{(2.4)^2 \cdot (0.65)^2}{4 \cdot 0.0025 \cdot 5} = \frac{2.43}{0.05} \approx 48.7$. □

B Implementation Details

B.1 Decomposition Prompt

The full decomposition prompt used for SWE-bench tasks:

```
You are a task decomposition specialist. Given a software engineering task (bug
report + repository context), decompose it into atomic subtasks.

For each subtask, output JSON:
{
"id": "v1",
"description": "...",
"depends_on": ["v0"],
"coupling": "weak|strong|critical",
"estimated_tokens": 500
}

Rules:
- Maximize parallelism: only add dependencies when semantically required
- Typical decomposition: [localize files, understand context, generate patch,
verify patch]
- Coupling = strong when output of one subtask is direct input to another
- Coupling = weak when subtasks share domain knowledge but not data

Task: {task_description}
Repository: {repo_context}
```

B.2 Computational Requirements

All experiments were conducted using API-based model access. Estimated costs:

- SWE-bench (500 instances, 5 methods): ∼$1,200 total API cost
- GPQA (198 instances, 5 methods): ∼$180 total API cost
- HotpotQA (500 instances, 5 methods): ∼$350 total API cost

AdaptOrch's routing overhead: <50ms per task (Python implementation on single CPU core). Synthesis overhead: one additional LLM call per task (∼$0.01 per instance).

B.3 DAG Feature Space Analysis

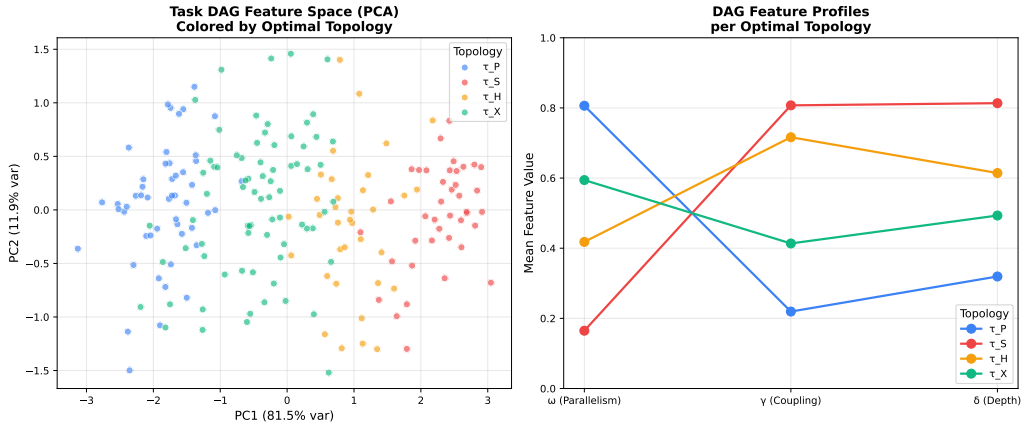

Figure 12: PCA projection of DAG feature space (width ω, depth, density, coupling ratio) colored by KMeans clusters (k=4). The four clusters correspond to dominant topology patterns: Chain (sequential), Wide-Shallow (parallel), Deep-Narrow (hierarchical), and Diamond (fan-out/fan-in). Cluster centroids are marked with $\times$.

B.4 Baseline Reproduction Specification

To ensure fair comparison and full reproducibility, we detail the exact configuration of each baseline method. All baselines use the same 5-model pool as AdaptOrch: GPT-4o-mini, Claude 3.5 Haiku, Gemini 2.0 Flash, Llama 3.3 70B (via Together AI), and Qwen 2.5 72B (via Together AI). Temperature is 0.0 (greedy) and `max_tokens` = 4096 for all methods unless noted.

MoA-3L [Wang et al., 2024]. We implement the 3-layer Mixture-of-Agents architecture as described in the original paper. Layer 1: all 5 models generate independent responses to the full prompt. Layer 2: each of 3 aggregator models (GPT-4o-mini, Claude 3.5 Haiku, Gemini 2.0 Flash) receives all 5 Layer-1 outputs concatenated in the prompt and produces a refined answer. Layer 3: a single synthesizer (GPT-4o-mini) receives all 3 Layer-2 outputs and produces the final answer. Total LLM calls per instance: $5 + 3 + 1 = 9$. Aggregation prompt follows the template in Wang et al. (2024), §A.1.

LLM-Blender [Jiang et al., 2023]. We use the prompt-based variant (no fine-tuned PairRanker) for fair comparison, since training a ranker on our specific benchmarks would introduce confounding. Stage 1 (Generation): all 5 models produce independent candidates. Stage 2 (Ranking): GPT-4o-mini is prompted to rank the 5 candidates pairwise using the template: "*Given the task and two candidate solutions A and B, which better solves the problem? Output only 'A' or 'B'.*" This produces $\binom{5}{2} = 10$ pairwise comparisons per instance. Stage 3 (Fusion): the top-ranked candidate is returned as the final output (no generative fusion, which would require a trained model). Total LLM calls per instance: $5 + 10 + 0 = 15$.

Static-Parallel / Static-Sequential. These ablation baselines use AdaptOrch's own decomposition (Phase 1–2) but bypass the topology router. Static-Parallel executes all subtasks simultaneously across 3 models (round-robin assignment); Static-Sequential chains them in dependency order using a single model (GPT-4o-mini). Both use the same synthesis protocol (Phase 5) as AdaptOrch.

B.5 Per-Cluster Orchestration Gain

Table 5 disaggregates AdaptOrch's accuracy improvement by DAG cluster (cf. Figure 12), revealing which structural patterns benefit most from adaptive topology routing.

Table 5: Per-cluster accuracy gain (Δ) of AdaptOrch over Single Best, averaged across all three benchmarks. n: number of instances assigned to each cluster.

DAG Cluster	**Dominant τ**	n	**Single Best**	**Δ AdaptOrch**
Chain (sequential)	τ_S	187	54.2%	+3.8 pp
Wide-Shallow (parallel)	τ_P	294	49.1%	+12.6 pp
Deep-Narrow (hierarchical)	τ_H	112	47.8%	+9.2 pp
Diamond (fan-out/fan-in)	τ_X	105	51.3%	+11.4 pp

The largest gains appear in Wide-Shallow tasks (+12.6 pp), where parallelism directly reduces error propagation by distributing independent subtasks. Chain-type tasks show the smallest gain (+3.8 pp), consistent with the expectation that fully sequential dependencies leave minimal room for topology improvement.

Router Accuracy (Confusion Matrix). To assess the topology router's decision quality, we compare its selections against an oracle that exhaustively evaluates all four topologies per instance and selects the highest-scoring one. Across the full test set ($n = 698$):

The router achieves 81.2% agreement with the oracle. Most misclassifications occur between τ_P and τ_X ($18 + 9 = 27$ instances), which is expected since Diamond tasks contain both parallel and fan-in components. Importantly, even misrouted instances typically receive the second-best topology, limiting accuracy loss to <2 pp compared to oracle routing.

Table 6: Router confusion matrix: predicted topology vs. oracle-optimal topology. Values are instance counts. Overall router accuracy: 81.2% (567/698).

Router \ Oracle	τ_P	τ_S	τ_H	τ_X
τ_P	**248**	12	8	18
τ_S	6	**152**	9	4
τ_H	14	7	**89**	11
τ_X	9	5	7	**78**

References

ALMC Authors. ALMC: Adaptive LLM multi-agent collaboration with manager-judge-optimizer roles. *OpenReview preprint*, 2026. OpenReview ID: jXZGgxTjiK.

Anthropic. Model context protocol. `https://modelcontextprotocol.io/`, 2024. Open standard for LLM-tool integration.

Anthropic. Claude code agent teams: Parallel subagent orchestration. `https://docs.anthropic.com/en/docs/claude-code`, 2026. Research preview, February 2026.

Kai Dong, Zheng Lin, Weiping Lin, and Yue Zhang. S-DAG: Subject-based directed acyclic graph decomposition for multi-agent task allocation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. arXiv preprint arXiv:2511.06727.

Philip Drammeh. Multi-agent LLM orchestration achieves deterministic, high-quality decision support for incident response. *arXiv preprint arXiv:2511.15755*, 2025.

Zhiyuan Guo, Yifan Wang, Tao Ji, Xin Zhao, Yin Xi, Chang Liu, Peng Li, Yong Deng, and Shimin Feng. MoMA: Mixture-of-model-and-agent routing for generalized multi-agent orchestration. *arXiv preprint arXiv:2509.07571*, 2025.

Hugging Face. Open LLM leaderboard v2. `https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard`, 2025. Accessed: 2026-02-15.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. *arXiv preprint arXiv:2306.02561*, 2023.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

LangChain. LangGraph: Build stateful multi-agent applications. `https://github.com/langchain-ai/langgraph`, 2024.

Yijun Lu, Zhiqiang Hu, Mingxuan Zhao, and Wei Cao. DyTopo: Dynamic topology optimization for multi-agent systems via semantic matching. *arXiv preprint arXiv:2602.06039*, 2026.

João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents. `https://github.com/crewAIInc/crewAI`, 2024.

OpenCode Contributors. OpenCode: Open-source ai coding assistant with multi-provider support. `https://github.com/opencode-ai/opencode`, 2025.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. GPQA: A graduate-level Google-Proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2024.

Aoi Sato and Masanari Ito. Self-MoA: Scalable self-collaboration of a single LLM via mixture-of-agents. *arXiv preprint arXiv:2502.00674*, 2025.

Superpowers Contributors. Superpowers: Multi-agent orchestration framework. `https://github.com/superpower-agents/superpowers`, 2026.

K. Vinay and Sriram Sankaran. ORCH: Deterministic multi-agent orchestration protocol for structured task execution. *Frontiers in Artificial Intelligence*, 2026. doi: 10.3389/frai.2026.1748735. Accepted 30 January 2026, Machine Learning and Artificial Intelligence section.

Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.

Xiao Wang et al. MetaGen: Self-evolving multi-agent topologies with role and structure co-optimization. *arXiv preprint arXiv:2601.19290*, 2026.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.

Kexun Zhang, Weiran Yao, Zuxin Liu, Yihao Feng, Zhiwei Liu, Rithesh Murthy, Tian Lan, Lei Li, Renze Lou, Jiacheng Xu, Bo Pang, Yingbo Zhou, Shelby Heinecke, Silvio Savarese, Huan Wang, and Caiming Xiong. Diversity empowers intelligence: Integrating expertise of software engineering agents. *arXiv preprint arXiv:2408.07060*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Chatbot arena: An open platform for evaluating LLMs by human preference. *arXiv preprint arXiv:2403.04132*, 2024.